

deque 정리

앞과 뒤 양쪽에서 빠르게 삽입/삭제할 수 있는 컨테이너입니다.

사용처 (Where)

- 앞에서도 넣고 빼야 할 때
- 슬라이딩 윈도우
- queue/stack보다 유연한 양방향 처리
- 맨 앞 삽입이 잦은 경우

방법 (How to Use)

- push_back/push_front로 양쪽 추가
- pop_back/pop_front로 양쪽 제거
- 인덱스 접근 가능
- front/back으로 양끝 확인

기본 정보

- 필요 헤더: #include <deque>
- 기본 선언: deque<int> dq;

왜 써야 할까? (Why)

- vector보다 앞쪽 삽입/삭제에 강함
- queue와 stack의 기반으로 이해하기 좋음
- 양방향 처리가 필요한 문제에서 코드가 단순해짐

주요 함수

함수/문법	의미
push_back/front	뒤/앞 추가
pop_back/front	뒤/앞 제거
front()/back()	양끝 접근
operator[]	인덱스 접근
size()	개수

기본 코드 예시

```
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> dq;
    dq.push_back(2);
    dq.push_front(1);
    dq.push_back(3);

    for (int x : dq) cout << x << ' ';
}
```

deque 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
양끝 삽입/삭제	$O(1)$
인덱스 접근	$O(1)$
중간 삽입/삭제	$O(n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 앞뒤로 명령 처리

명령에 따라 앞 또는 뒤에 값을 넣고 빼는 형태의 문제에 적합합니다.

```
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> dq = {2, 3};
    dq.push_front(1);
    dq.push_back(4);
    dq.pop_front();

    cout << dq.front() << ' ' << dq.back();
}
```

처음 배울 때 자주 하는 실수

- vector처럼 메모리가 완전히 연속이라는 보장은 약함
- 중간 삽입/삭제가 많으면 list도 검토
- 단순 뒤쪽 추가만 많으면 vector가 더 익숙하고 좋음

비슷한 항목과 차이

- vector는 뒤쪽 추가와 인덱스 접근에 강함
- list는 중간 삽입/삭제에 강함
- queue는 deque를 제한적으로 쓰는 형태

한 줄 정리: deque은(는) '앞에서도 넣고 빼야 할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.